

CyberAudit

Mock Data Import

35 audit requests + 9 client accounts, localStorage to PostgreSQL

Project	CyberAudit Platform
File	cyberaudit/backend/import_mock_data.py
Source data	localStorage mock data from the frontend prototype
Target	PostgreSQL via Django ORM (manage.py shell)
Clients imported	9 accounts (get_or_create + set_password)
Requests imported	35 AuditRequests, DOSSIER-2026-0001 to 0035
Idempotent	Re-running skips existing references (filter().exists() guard)
submitted_at fix	.update() bypass -- auto_now_add cannot be set via assignment
Run command	python manage.py shell < import_mock_data.py
Date	June 2026 / Juin 2026

1. Context & Purpose

1. Contexte et objectif

The CyberAudit frontend was built first as a prototype using localStorage as a mock database. When the Django/PostgreSQL backend went live, 35 historical audit requests from 9 client accounts needed to be migrated from that mock data into the real database, without losing timestamps or creating duplicates.

Le frontend CyberAudit a d'abord été construit comme un prototype utilisant localStorage comme base de données fictive. Quand le backend Django/PostgreSQL est entré en production, 35 demandes d'audit historiques de 9 comptes clients ont dû être migrées depuis ces données fictives vers la vraie base, sans perdre les horodatages ni créer de doublons.

```
'''
import_mock_data.py -- Imports 35 requests from localStorage into Django.

Run with:
  cd ~/portfolio/cyberaudit/backend
  python manage.py shell < import_mock_data.py
'''

import os, django
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
django.setup() # initialise Django outside of runserver

from apps.accounts.models import User
from apps.audits.models import AuditPack, AuditRequest
```

manage.py shell < file	Runs the script inside the Django shell context. ORM, settings, and apps are all available. No custom management command needed. Exécute le script dans le contexte Django shell. ORM, settings et apps sont disponibles. Aucune commande de gestion personnalisée nécessaire.
django.setup()	Required when running a standalone script (not via manage.py). Initialises the app registry so models can be imported. Requis pour un script autonome. Initialise le registre des apps pour que les modèles soient importables.
Why not a data migration?	Data migrations run at deploy time and are version-controlled. A one-off import script is simpler and appropriate for a single historical batch. Les migrations de données s'exécutent au déploiement. Un script ponctuel est plus simple pour un import historique unique.

2. Step 1: Load Packs (pack lookup dict)

2. Etape 1: Chargement des packs (dict de lookup)

Before inserting any request, all AuditPack rows are loaded into a dictionary keyed by code. This avoids N individual SELECT queries, the pack lookup for all 35 requests costs exactly 1 query.

Avant d'insérer toute demande, toutes les lignes AuditPack sont chargées dans un dictionnaire indexé par code. Cela évite N requêtes SELECT individuelles, le lookup de pack pour les 35 demandes ne coûte exactement qu'1 requête.

```
# Step 1: load all packs in 1 query
packs = {p.code: p for p in AuditPack.objects.all()}
print(f'Packs trouvés : {list(packs.keys())}')

# Expected output:
# Packs trouvés : ['audit', 'security', 'protection', 'premium']
```

Dict comprehension	O(1) lookup per request instead of O(N) DB queries. 1 SELECT for all 4 packs total. O(1) lookup par demande au lieu de O(N) requêtes DB. 1 SELECT pour les 4 packs.
Early validation	If packs dict is empty (0002_seed_packs not yet run), every request row will be skipped with a warning rather than crashing. Si le dict est vide, chaque ligne est ignorée avec un avertissement plutôt qu'un crash.

3. Step 2 : Create 9 Client Accounts

3. Etape 2 : Creation de 9 comptes clients

9 client accounts are created with `get_or_create`, idempotent by email. The script sets a temporary hashed password via `set_password()` so each account is immediately usable. A `client_map` dict is built for the request-import step.

9 comptes clients sont créés avec `get_or_create`, idempotent par email. Le script définit un mot de passe hache temporaire via `set_password()` pour que chaque compte soit immédiatement utilisable. Un dict `client_map` est construit pour l'étape d'import des demandes.

```
CLIENTS = {
    'marie@cabinet-dijon.fr': ('Marie', 'Dupont', 'Cabinet Dijon'),
    'p.mercier@dijon.fr': ('Paul', 'Mercier', 'City Hall Dijon'),
    'contact@atelier-pro.fr': ('Lucie', 'Robert', 'Atelier Pro'),
    'sami@techcorp.io': ('Sami', 'Benali', 'Tech Corp'),
    'emma@bluerooft.fr': ('Emma', 'Leroy', 'BlueRoof SARL'),
    'hugo@boulangerie-martin.fr': ('Hugo', 'Martin', 'Boulangerie Martin'),
    'tomjouan99@gmail.com': ('Tommy', 'Jouhans', 'Holberton School
Dijon'),
    'tommy.jouhans@outlook.com': ('Tommy', 'Jouhans', 'Holberton School
Dijon'),
    'jean@test.com': ('Jean', 'Dupont', 'Holberton School
Dijon'),
}

client_map = {} # email -> User instance
for email, (first, last, company) in CLIENTS.items():
    user, created = User.objects.get_or_create(
        email=email,
        defaults={'first_name': first, 'last_name': last,
                  'company_name': company, 'role': 'client'},
    )
    if created:
        user.set_password('Client1234!') # hashes via PBKDF2
        user.save()
    client_map[email] = user
```

9 clients accounts

Email	Name	Company	Note EN / FR
marie@cabinet-dijon.fr	Marie Dupont	Cabinet Dijon	Fictional SME client. First 4 requests. Client PME fictif. 4 premières demandes.
p.mercier@dijon.fr	Paul Mercier	City Hall Dijon	Public sector. Continuous monitoring. Secteur public. Surveillance continue.
contact@atelier-pro.fr	Lucie Robert	Atelier Pro	Archived. Earliest test client. Archivée. Premier client test.
sami@techcorp.io	Sami Benali	Tech Corp	Startup. First audit. Startup. Premier audit.
emma@blueroof.fr	Emma Leroy	BlueRoof SARL	Post-phishing incident. Après incident de phishing.
hugo@boulangerie-martin.fr	Hugo Martin	Boulangerie Martin	3 shops, premium pack. 3 boutiques, pack premium.
tomjouan99@gmail.com	Tommy Jouhans	Holberton School Dijon	Developer test account. 7 requests. Compte test développeur. 7 demandes.
tommy.jouhans@outlook.com	Tommy Jouhans	Holberton School Dijon	Primary test account. 17 requests. Compte test principal. 17 demandes.
jean@test.com	Jean Dupont	Holberton School Dijon	QA test account. 4 requests. Compte QA. 4 demandes.
get_or_create	Lookup by email, the unique login field. If the user already exists (e.g., from a previous import run), the existing row is returned and no password is reset.		

	Lookup par email, le champ de connexion unique. Si l'utilisateur existe déjà, la ligne existante est retournée et le mot de passe n'est pas remis à zéro.
set_password()	Hashes the password using Django's PBKDF2-SHA256 hasher. Never stores plain text. The 'Client1234!' default can be changed via admin. Hache le mot de passe avec PBKDF2-SHA256. Jamais de texte clair. Le défaut 'Client1234!' peut être change via l'admin.
role: client	All imported accounts are clients. Admin accounts are created separately via createsuperuser. Tous les comptes importés sont clients. Les comptes admin sont créés séparément via createsuperuser.
client_map dict	email -> User instance. Built once, reused for all 35 request rows. No per-row DB lookup for the FK. email -> instance User. Construit une fois, réutilise pour les 35 demandes. Pas de lookup DB par ligne.

4. Step 3 -- Import Logic & submitted_at Bypass

4. Etape 3 -- Logique d'import et contournement de submitted_at

Each of the 35 rows is checked for existence before insertion. The most technically interesting part is the submitted_at bypass: because the field uses auto_now_add=True, Django ignores any assignment made in Python. The workaround is a targeted .update() call which writes directly to the DB row, bypassing the field's auto-stamp.

Chacune des 35 lignes est vérifiée avant insertion. La partie techniquement la plus intéressante est le contournement de submitted_at : le champ utilisant auto_now_add=True, Django ignore toute assignation en Python. La solution est un appel .update() cible qui écrit directement dans la ligne DB, contournant l'horodatage automatique.

```
created_count = 0
skipped_count = 0

for reference, email, pack_code, status, notes, submitted_iso in REQUESTS:

    # Guard: skip if already imported
    if AuditRequest.objects.filter(reference=reference).exists():
        print(f'  ⏮ Déjà en base : {reference}')
        skipped_count += 1
        continue

    client = client_map.get(email)
    pack = packs.get(pack_code)
    if not client or not pack:
        print(f'  ✖ Ignoré {reference} - client ou pack manquant')
        continue

    # Create row (reference set manually, NOT via pre_save signal)
    r = AuditRequest(
        reference=reference,
        client=client,
        pack=pack,
        status=status,
        scope_notes=notes,
    )
    r.save()

    # Fix submitted_at -- auto_now_add blocks normal Python assignment
    from datetime import datetime as dt
    submitted_dt = dt.fromisoformat(submitted_iso.replace('Z', '+00:00'))
    AuditRequest.objects.filter(pk=r.pk).update(submitted_at=submitted_dt)

    print(f'  ☑ {reference} | {status:12} | {pack_code:10} | {email}')
    created_count += 1

print(f'Import terminé : {created_count} créés, {skipped_count} ignorés')
print(f'Total en base : {AuditRequest.objects.count()} demandes')
```

The auto_now_add bypass explained

`auto_now_add=True` tells Django to set the field to `now()` at INSERT time and to mark it as non-editable. Any Python assignment like `r.submitted_at = some_date` is silently ignored before the INSERT SQL is built. The `.update()` queryset method bypasses this -- it generates an UPDATE SQL directly on the already-inserted row and does not go through the model's field-level validation.

auto_now_add=True dit à Django de définir le champ à `now()` à l'INSERT et de le marquer comme non modifiable. Toute assignation Python comme `r.submitted_at = some_date` est silencieusement ignorée avant la construction du SQL INSERT. La méthode `.update()` du queryset contourne cela, elle génère un SQL UPDATE directement sur la ligne déjà insérée sans passer par la validation au niveau du champ.

```
# ✗ WRONG -- silently ignored because auto_now_add=True marks field
# editable=False
r.submitted_at = submitted_dt
r.save() # submitted_at will be Django's 'now', not the historical date

# ☑ CORRECT -- .update() writes to DB directly, bypasses model field
# restrictions
AuditRequest.objects.filter(pk=r.pk).update(submitted_at=submitted_dt)

# ISO-8601 parse: 'Z' -> '+00:00' makes it timezone-aware (UTC)
submitted_dt = dt.fromisoformat(submitted_iso.replace('Z', '+00:00'))
```

filter(reference=...).exists()	1 SELECT COUNT query per row. Returns True immediately if found - no full row fetch. 1 requête SELECT COUNT par ligne. Retourne True immédiatement si trouve.
reference set manually	Normally set by a <code>pre_save</code> signal. The import bypasses it by assigning directly, the unique constraint still enforces no collisions. Normalement défini par un signal <code>pre_save</code>. L'import l'assigne directement ; la contrainte unique empêche les collisions.
.update() post-INSERT	Two-step: INSERT (auto now), then UPDATE (historical date). Clean and explicit. Deux étapes : INSERT (now auto), puis UPDATE (date historique). Propre et explicite.
Timezone-aware datetimes	<code>USE_TZ=True</code> in Django settings. All datetimes must be timezone-aware. The <code>'Z'-'>'+00:00'</code> replacement ensures correct UTC parsing via <code>fromisoformat</code>. <code>USE_TZ=True</code>. Tous les datetimes doivent être aware. Le remplacement <code>'Z'-'>'+00:00'</code> assure un parsing UTC correct.

5. The 35 Requests : Full Dataset

5. Les 35 demandes : Jeu de données complet

The 35 requests span April 25 to May 28 2026 and cover all 4 pack types and all 4 statuses. Most were generated during the developer testing phase in late May 2026.

Les 35 demandes couvrent du 25 avril au 28 mai 2026 et couvrent les 4 types de packs et les 4 statuts. La plupart ont été générées pendant la phase de test développeur fin mai 2026.

Reference	Client email	Pack	Status	Date (UTC)
0001	marie@cabinet-dijon.fr	premium	in_progress	2026-04-29
0002	marie@cabinet-dijon.fr	audit	pending	2026-04-22
0003	marie@cabinet-dijon.fr	security	completed	2026-04-10
0004	p.mercier@dijon.fr	protection	in_progress	2026-04-03
0005	contact@atelier-pro.fr	audit	archived	2026-03-25
0006	sami@techcorp.io	audit	pending	2026-04-18
0007	emma@blueroof.fr	security	completed	2026-04-12
0008	hugo@boulangerie-martin.fr	premium	pending	2026-05-05
0009	tomjouan99@gmail.com	audit	in_progress	2026-05-21
0010	tommy.jouhans@outlook.com	protection	archived	2026-05-21
0011	tommy.jouhans@outlook.com	audit	pending	2026-05-25
0012	jean@test.com	security	pending	2026-05-25
0013	jean@test.com	security	completed	2026-05-26
0014	jean@test.com	protection	pending	2026-05-26
0015	jean@test.com	audit	pending	2026-05-26

0016-0035	tommy.jouhans@outlook.com / tomjouan99	audit/security/protection	mostly pending	2026-05- 27 -- 28
-----------	---	---------------------------	-------------------	----------------------

Distribution by pack / status

By pack	Count	By status	Count
audit	20	pending	25
security	7	in_progress	4
protection	6	completed	4
premium	2	archived	2
TOTAL	35	TOTAL	35

6. Output & Idempotency

6. Sortie et idempotence

The script prints a status line for every row processed and a summary at the end. Running it a second time produces only 'skip' lines, no duplicates, no crashes. The final count query confirms the total in the database.

Le script affiche une ligne de statut pour chaque ligne traitée et un resume a la fin. Une deuxième execution ne produit que des lignes 'ignore', pas de doublons, pas de crash. La requête de comptage finale confirme le total en base.

```
# First run output:
Packs trouvés : ['audit', 'security', 'protection', 'premium']
  [x] DOSSIER-2026-0001 | in_progress | premium | marie@cabinet-dijon.fr
  [x] DOSSIER-2026-0002 | pending    | audit   | marie@cabinet-dijon.fr
  ...
  [x] DOSSIER-2026-0035 | pending    | audit   | tommy.jouhans@outlook.com

=====
Import terminé : 35 créés, 0 ignorés
Total en base  : 35 demandes

# Second run output (idempotent):
  [x] Déjà en base : DOSSIER-2026-0001
  [x] Déjà en base : DOSSIER-2026-0002
  ...
Import terminé : 0 créés, 35 ignorés
Total en base  : 35 demandes (unchanged)
```

filter().exists() guard	One SELECT COUNT(*) per row. Much faster than get() which fetches the full row on hit. Un SELECT COUNT(*) par ligne. Beaucoup plus rapide que get() qui recupere toute la ligne.
No transaction wrapping	Each row is committed independently. A mid-run failure leaves already-imported rows intact. Re-run skips them. Chaque ligne est commitee independamment. Un echec en cours laisse les lignes deja importees. La re-execution les ignore.
Final count query	AuditRequest.objects.count() -- confirms total in DB regardless of how many were created this run. Confirme le total en base independamment du nombre cree lors de cette execution.

7. Client Distribution by Request Count

7. Distribution des clients par nombre de demandes

The 35 requests are unevenly distributed, two developer test accounts (tommy.jouhans@outlook.com and tomjouan99@gmail.com) account for 24 of the 35 requests. This reflects the testing workflow: the developer submitted many requests to validate statuses, pack selection, and admin dashboard behaviour.

Les 35 demandes sont inégalement réparties, deux comptes de test développeur (tommy.jouhans@outlook.com et tomjouan99@gmail.com) représentent 24 des 35 demandes. Cela reflète le workflow de test : le développeur a soumis de nombreuses demandes pour valider les statuts, la sélection de packs et le comportement du tableau de bord admin.

Email	Company	Requests	References
tommy.jouhans@outlook.com	Holberton School Dijon	17	0010-0011, 0016-0029, 0034-0035
tomjouan99@gmail.com	Holberton School Dijon	7	0009, 0025, 0030-0033
marie@cabinet-dijon.fr	Cabinet Dijon	4	0001-0003, 0024
jean@test.com	Holberton School Dijon	4	0012-0015
p.mercier@dijon.fr	City Hall Dijon	1	0004
contact@atelier-pro.fr	Atelier Pro	1	0005
sami@techcorp.io	Tech Corp	1	0006
emma@blueroof.fr	BlueRoof SARL	1	0007
hugo@boulangerie-martin.fr	Boulangerie Martin	1	0008
TOTAL	9 clients	35	0001 - 0035

8. Technical Summary

8. Récapitulatif technique

Pattern	Detail EN / FR
manage.py shell <	Runs the script inside the fully-initialised Django environment. No custom management command or migration needed. Exécute le script dans l'environnement Django initialise. Aucune commande personnalisée ou migration nécessaire.
Idempotency	filter(reference=...). exists() guard before every INSERT. Safe to re-run multiple times, produces 0 duplicates. Guard filter().exists() avant chaque INSERT. Sécurise pour plusieurs exécutions, 0 doublon.
get_or_create clients	Email is the unique key. Existing accounts are returned as-is; their passwords are not reset. Email est la clé unique. Les comptes existants sont retournés tels quels ; mots de passe non réinitialisés.
set_password()	Always hashes via PBKDF2-SHA256. Never stores plain text. Django auth system is fully respected. Hache toujours via PBKDF2-SHA256. Jamais de texte clair. Systeme d'auth Django pleinement respecte.
Pack dict (1 query)	All 4 packs loaded in 1 SELECT. O(1) dict lookup per request row. No N+1 for FK resolution. 4 packs chargés en 1 SELECT. Lookup O(1) par ligne. Pas de N+1 pour la résolution FK.
client_map dict	All 9 users held in memory after get_or_create. 0 additional DB queries for client FK resolution. 9 utilisateurs en mémoire après get_or_create. 0 requête DB supplémentaire pour le FK client.
.update() bypass	auto_now_add=True prevents Python assignment. .update() sends UPDATE SQL directly, bypassing field validation. auto_now_add=True empêche l'assignation Python. .update() envoie l'UPDATE SQL directement.
Timezone-aware ISO	'Z' -> '+00:00' makes fromisoformat() produce a UTC-aware datetime. Required for Django USE_TZ=True. 'Z' -> '+00:00' produit un datetime aware UTC. Requis pour USE_TZ=True.
Manual reference	Bypasses the pre_save signal that auto-generates DOSSIER-YYYY-NNNN. Unique constraint still enforced.

	Contourne le signal pre_save qui auto-génère la référence. Contrainte unique toujours active.
No transaction block	Each row commits independently. Mid-run crash is safe, resume from the skipped rows. Chaque ligne commite indépendamment. Un crash en cours est sûr, reprise depuis les lignes ignorées.